Network Mapper and Secure Shell Operations

CYBERSECURITY

MAHBOUBA REZAEI - REZ23579670
UNIVERSITY OF ROEHAMPTON
30 MARCH 2026

GITHUB LINK: https://github.com/M12rezaei/CyberSecurity.git

Contents

Basic Host Discovery with Nmap 3
1.0 Basic Host Discovery 3
1.1 Objective 3
1.2 Methodology and Findings 3
Reflection 4
2.0 Port Scanning and Service Detection 5
2.1 Objective 5
2.2 Methodology and Findings 5
UDP Scanning 6
Targeted Port Ranges 6
3.0 Service Version Detection 7
3.1 Objective 7
3.2 Methodology and Findings 7
4.0 Operating System (OS) Detection 8
4.1 Objective 8
4.2 Methodology and Findings 8
5.0 Aggressive Scanning 9
5.1 Objective 9
5.2 Methodology and Finding 9
6.0 Nmap Scripting Engine (NSE) 10
6.1 Objective 10
6.2 Methodology and Findings 10
7.0 Output Options 12
7.1 Objective 12
7.2 Methodology and Findings 12
8.0 Advanced Scanning Techniques 13
8.1 Objective 13
8.2 Methodology and Findings 13
9.0 Other Useful Commands 16
9.1 Objective 16
9.2 Methodology and Findings 16
Final Reflection 18
Secure Shell (SSH) 19
SSH Connection Basics 19
Objective 19

Methodology and Findings 19
2.0 Authentication and Key Management 21
2.1 Objective 21
2.2 Methodology and Findings 21
3.0 Managing SSH Sessions 22
3.1 Objective 22
3.2 Methodology and Findings 22
4.0 File Transfer with SCP (Secure Copy) 23
4.1 Objective 23
4.2 Methodology and Findings 23
5.0 Secure File Transfer with SFTP 24
5.1 Objective 24
5.2 Methodology and Findings 24
6.0 Tunneling with SSH 24
6.1 Objective 24
6.2 Methodology and Findings 24
7.0 Multiplexing and Persistent Connections 26
7.1 Objective 26
7.2 Methodology and Findings 26
8.0 Advanced SSH Options and Debugging 27
8.1 Objective 27
8.2 Methodology and Findings 27
Final Reflection 29
References 30

Basic Host Discovery with Nmap

1.0 Basic Host Discovery

1.1 Objective

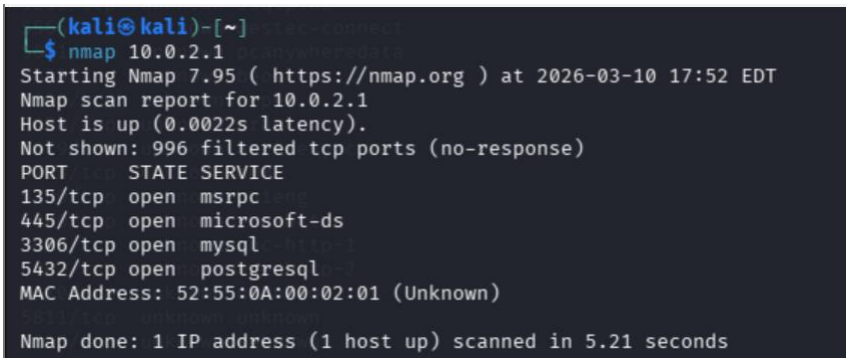
The aim of this exercise was to identify live hosts on the network segment using various Nmap scanning techniques. This is a fundamental step in reconnaissance, allowing us to map out reachable devices before performing deeper vulnerability analysis.

1.2 Methodology and Findings

Single Host Scan:

I began by targeting the specific host 10.0.2.1, to determine its status

- **Command:** nmap 10.0.2.1
- **Finding:** the host was confirmed as 'up'. The scan revealed several open services, including the following services:
  - 135/tcp (msrpc),
  - 445/tcp (Microsoft-ds),
  - 3306/tcp (mysql), and
  - 5432/tcp (postgresql)



```
┌──(kali㉿kali)-[~]
└─$ nmap 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:52 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0022s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.21 seconds
```

Figure 1. Nmap scan results for a single host (10.0.2.1) identifying active status and specific open ports for MSRPC, SMB, MySQL, and PostgreSQL

Multiple Host Scan

To simulate checking multiple assets simultaneously, I combined the targets into a single command

- **Command:** nmap 10.0.2.1 10.0.2.15
- **Finding:** this scan successfully confirmed that both 10.0.2.1 and 10.0.2.15 were active on the network. This provides a side-by-side view of their security posture, noting that 10.0.2.15 (the local machine) showed no open ports, contrasting with the active services found on 10.0.2.1 [1].

```
┌──(kali㉿kali)-[~]
└─$ nmap 10.0.2.1 10.0.2.15
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:53 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0018s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap scan report for 10.0.2.15
Host is up (0.0000030s latency).
All 1000 scanned ports on 10.0.2.15 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 2 IP addresses (2 hosts up) scanned in 5.29 seconds
```

Figure 2. Nmap scan result for multiple hosts

Subnet Scan

- **Command:** nmap 10.0.2.1/24
- **Finding:** The scan identified three active hosts: 10.0.2.1, 10.0.2.2, and 10.0.2.15. It confirmed that the network environment is populated by these specific nodes, providing a clear map of the local infrastructure.

```
┌──(kali㉿kali)-[~]
└─$ nmap 10.0.2.1/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:55 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0023s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap scan report for 10.0.2.2
Host is up (0.00068s latency).
All 1000 scanned ports on 10.0.2.2 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:D5:1D:7D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 10.0.2.15
Host is up (0.000048s latency).
All 1000 scanned ports on 10.0.2.15 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (3 hosts up) scanned in 7.23 seconds
```

Figure 3. Nmap subnet scan results, identifying three active hosts and providing a comprehensive map of the local network

Reflection

Performing these discovery scans highlighted the importance of establishing a baseline for the network. Initially, I attempted to scan my local machine (10.0.2.15), but I found that it appeared secure with no open ports. Switching the focus to 10.0.2.1 provided a much more realistic scenario, as it allowed me to identify running services that could potentially be audited for vulnerabilities. This exercise demonstrated that host discovery is not just about identifying

active hosts, but also about identifying which targets are worth investing further based on their exposed services.

2.0 Port Scanning and Service Detection

2.1 Objective

Following the discovery of live hosts, the next activity in the lab sheet required a more detailed investigation into the ports of the target machine (10.0.2.1). The goal was to compare different scanning methods, ranging from stealthy techniques to protocol specific searches, to see how they interact with the target's security posture.

2.2 Methodology and Findings

TCP Connect and SYN Stealth Scans:

I performed both a TCP Connect scan (-sT) and a SYN scan (-sS) as directed by the lab activities. These scans rely on the Transmission Control Protocol (TCP), a connection-oriented protocol at the Transport Layer that ensures reliable data delivery through a Three-Way Handshake (SYN, SYN-ACK, ACK) [2][3].

- **Commands:** nmap -sT 10.0.2.1 and nmap -sS 10.0.2.1
- **Finding:** both methods confirmed that ports 135, 45, 3306, and 5432 were open. While the TCP Connect Scan is straightforward, the SYN scan is typically noted in the lab as being more "stealthy" because it doesn't complete the full three-way handshake, making it less likely to be logged by basic application-level monitors [1].

```
┌──(kali㉿kali)-[~]
└─$ nmap -sT 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:57 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0040s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.19 seconds
```

Figure 4. Execution of a TCP Connect Scan (-sT) on host 10.0.2.1

```
┌──(kali㉿kali)-[~]
└─$ nmap -sS 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:57 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0021s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.13 seconds
```

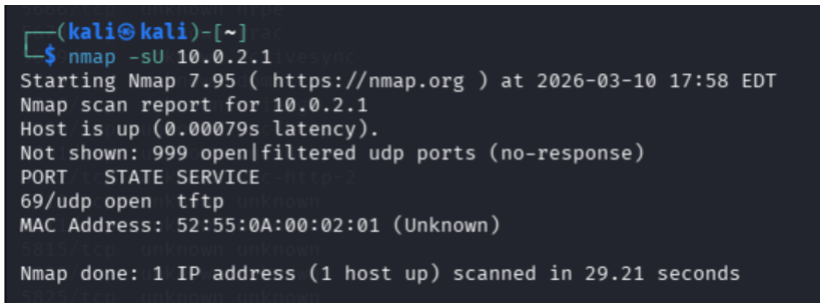
Figure 5. Implementing a SYN Stealth Scan (-sS) to identify open ports

- **Analysis:**
    - TCP Connect Scan (-sT): This scan completes the entire three-way handshake. While reliable, it is easily detected because the target system logs the fully established connection [2].
    - SYN Stealth Scan (-sS): This is often referred to as a “half-open” scan. Nmap sends a SYN packet, if it receives a response, it sends an RST (Reset) packet instead of the final ACK. This avoids completing the connection, meaning the attempt is often not logged by basic application-level monitors, providing a significant stealth advantage during reconnaissance [3].

UDP Scanning

The lab sheet highlighted that services often hide on UDP ports, which are frequently missed during standard TCP audits.

- **Command:** nmap -sU 10.0.2.1
- **Finding:** This activity was particularly useful as it identified port 69/udp as open, running the TFTP service. This result demonstrates why a comprehensive scan must include UDP to avoid leaving blind spots in the network map [1].
- **Analysis:** UDP scanning is technically challenging because it relies on the absence of a response to determine a port’s state. When Nmap sends a UDP packet and receives no response, it often reports the state as open|filtered, as it cannot distinguish between an open port and a packet dropped by a firewall [5].



```
┌──(kali㉿kali)-[~]
└─$ nmap -sU 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 17:58 EDT
Nmap scan report for 10.0.2.1
Host is up (0.00079s latency).
Not shown: 999 open|filtered udp ports (no-response)
PORT   STATE SERVICE
69/udp open  tftp
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 29.21 seconds
```

Figure 6. Performing a UDP scan (-sU) on 10.0.2.1

Targeted Port Ranges

I practised limiting the scope of a scan by targeting only the first 100 ports and then specific common ports.

- **Commands:** nmap -p 1-100 10.0.2.1 and nmap 22,80,443 10.0.2.1
- **Finding:** These scans returned “filtered” or “ignored” results. This was a valuable exercise because it showed that just because common ports like 80 (HTTP) are closed, it does not necessarily mean the host is secure, the actual active services were located elsewhere on the device. A closed port confirms to an attacker that a host is “up”, whereas a filtered port suggests the presence of a firewall or packet filter that is actively dropping probes [6].

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 1-100 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:00 EDT
Nmap scan report for 10.0.2.1
Host is up (0.00066s latency).
All 100 scanned ports on 10.0.2.1 are in ignored states.
Not shown: 100 filtered tcp ports (no-response)
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.90 seconds
```

Figure 7. Limiting the scan scope to the first 100 ports

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 22,80,443 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:01 EDT
Nmap scan report for 10.0.2.1
Host is up (0.00098s latency).

PORT    STATE    SERVICE
22/tcp  filtered ssh
80/tcp  filtered http
443/tcp filtered https
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 1.54 seconds
```

Figure 8. Targeting specific common administrative and web ports (22, 80, 443)

3.0 Service Version Detection

3.1 Objective

The activity in this section was to move beyond simply knowing a port is “open” and instead identify exactly what software is running on the service. This is a critical skill for vulnerability management.

3.2 Methodology and Findings

Version Detection (-sV)

The scan ran the version detection command against the target to pull specific metadata from the open services

- **Command:** nmap -sV 10.0.2.1
- **Finding:** This provided highly specific data that turned “open ports” into “identifiable targets”. For example, port 5432 was identified as “PostgreSQL DB 9.6.0 or later”. On port 3306, Nmap even provided a technical “fingerprint” because the service (MySQL) was using a modern authentication method (caching_sha2_password) that required extra analysis [1].

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:01 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0036s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE        VERSION
135/tcp  open  msrpc          Microsoft Windows RPC
445/tcp  open  microsoft-ds?
3306/tcp open  mysql?
5432/tcp open  postgresql     PostgreSQL DB 9.6.0 or later
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-serv
ice :
SF-Port3306-TCP:V=7.95%I=7%D=3/10%Time=69B094CD%P=x86_64-pc-linux-gnu%r(NU
SF:LL,4D,"I\0\0\0\n9\.6\.0\0\x18\0\0\0MBAGIk\?z\0\xff\xff\xff\x02\0\xff\xd
SF:f\x15\0\0\0\0\0\0\0\0\0\0\0\+\x02\x0cZoMyC\(;Z\x01\0caching_sha2_password
SF:\0")%r(GenericLines,72,"I\0\0\0\n9\.6\.0\0\x18\0\0\0MBAGIk\?z\0\xff\xff
SF:\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\0\0\0\0\0\+\x02\x0cZoMyC\(;Z\x01\0cach
SF:ing_sha2_password\0!\0\0\x01\xff\x84\x04#08S01Got\x20packets\x20out\x20
SF:of\x20order")%r(GetRequest,72,"I\0\0\0\n9\.6\.0\0\x19\0\0\0;&nAJ\[L\x03
SF:\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\0\0\0\x0f\x0e\"b40L3%a\x
SF:10A\0caching_sha2_password\0!\0\0\x01\xff\x84\x04#08S01Got\x20packets\x
SF:20out\x20of\x20order")%r(HTTPOptions,72,"I\0\0\0\n9\.6\.0\0\x1a\0\0\0\x
SF:17\x1c\x03\x1fs\x1cN\x0b\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\
SF:0\0\0i\x1c\x17G\x15\"\*8BK\r1\0caching_sha2_password\0!\0\0\x01\xff\x84
SF:\x04#08S01Got\x20packets\x20out\x20of\x20order")%r(RTSPRequest,72,"I\0\
SF:0\0\0n9\.6\.0\0\x1b\0\0\0\)TZ\|UXxB\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\
SF:0\0\0\0\0\0\0\0S\x03z!\x03\x0c,WVc\x1fm\0caching_sha2_password\0!\0\0\x
SF:01\xff\x84\x04#08S01Got\x20packets\x20out\x20of\x20order")%r(RPCCheck,7
SF:2,"I\0\0\0\n9\.6\.0\0\x1c\0\0\0\.L\x0fR'c7M\0\xff\xff\xff\x02\0\xff\xdf
SF:\x15\0\0\0\0\0\0\0\0\0\0\+,-\(Q<V\x1e\]\\x\x01\0caching_sha2_password\0
SF:!\0\0\x01\xff\x84\x04#08S01Got\x20packets\x20out\x20of\x20order")%r(DNS
SF:VersionBindReqTCP,72,"I\0\0\0\n9\.6\.0\0\x1d\0\0\0\x0b\x7f\|\x05o\*9{\0
SF:\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\0\0\0\0h\x1a\x1b@\x07'\x1aN/
SF:\.,Z\0caching_sha2_password\0!\0\0\x01\xff\x84\x04#08S01Got\x20packets\
SF:x20out\x20of\x20order")%r(DNSStatusRequestTCP,72,"I\0\0\0\n9\.6\.0\0\x1
SF:e\0\0\0\x1eX\x08aF,a#\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\0\0
SF:\0nQ\]dMz\x01YkWwf\0caching_sha2_password\0!\0\0\x01\xff\x84\x04#08S01G
SF:ot\x20packets\x20out\x20of\x20order")%r(Help,72,"I\0\0\0\n9\.6\.0\0\x1f
SF:\0\0\0\^h<\"\x06\x17v\x01\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\
SF:\0\0\0l-:4\)ZL\x01oqR1\0caching_sha2_password\0!\0\0\x01\xff\x84\x04#08
SF:S01Got\x20packets\x20out\x20of\x20order")%r(SSLSessionReq,72,"I\0\0\0\n
SF:9\.6\.0\0\x20\0\0\0qlH-\rS@>\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\
SF:0\0\0\0\0\[\.R%\]o\rYoW-\x17\0caching_sha2_password\0!\0\0\x01\xff\x84\
SF:x04#08S01Got\x20packets\x20out\x20of\x20order");
MAC Address: 52:55:0A:00:02:01 (Unknown)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.12 seconds
```

Figure 9. Executing service version detection (-sV) on 10.0.2.1

4.0 Operating System (OS) Detection

4.1 Objective

This activity required me to use Nmap's OS fingerprinting capabilities to identify the operating system running on the target host (10.0.2.1)

4.2 Methodology and Findings

- **Command:** nmap -O 10.0.2.1
- **Finding:** Nmap provided an "Aggressive OS guess", suggesting the target is likely an AT&T BGW210 voice gateway (94%) or an Oracle VirtualBox Slirp NAT bridge (90%)

```
┌──(kali㉿kali)-[~]
└─$ nmap -O 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:03 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0019s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: AT&T BGW210 voice gateway (94%), Oracle Virtualbox Slirp NAT bridge (90%), QEMU user mode network gateway (89%), HP 9100c Digital Sender printer
 (J3113A) (86%), Minolta Di550 laser printer (85%), NEC SuperScript printer (85%), Cisco SG 500 switch (85%), Yamaha RX-V2067 or RX-V3900 audio receiver (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.21 seconds
```

Figure 10. Nmap OS fingerprinting results showing an "Aggressive OS guess" for 10.0.2.1

- **Analysis:** According to official Nmap documentation, remote OS detection is achieved through TCP/IP stack fingerprinting. Nmap sends a series of TCP and UDP packets and examines "practically every bit in the responses," including TCP ISN sampling and IP ID sequence generation [7]. It then compares these unique patterns against a database of over 2,600 known fingerprints.

The “unreliable” warning in my results occurred because Nmap could not identify both open and closed ports simultaneously. As the source indicates, having both states is a prerequisite for the most accurate detection [7]. Without a closed port, Nmap must rely on “fuzzy” matching, resulting in the “Aggressive OS guess” and the percentage-based confidence levels (94% and 90%) seen in Figure 10. despite the target being directly adjacent to the Kali machine on the virtual network.

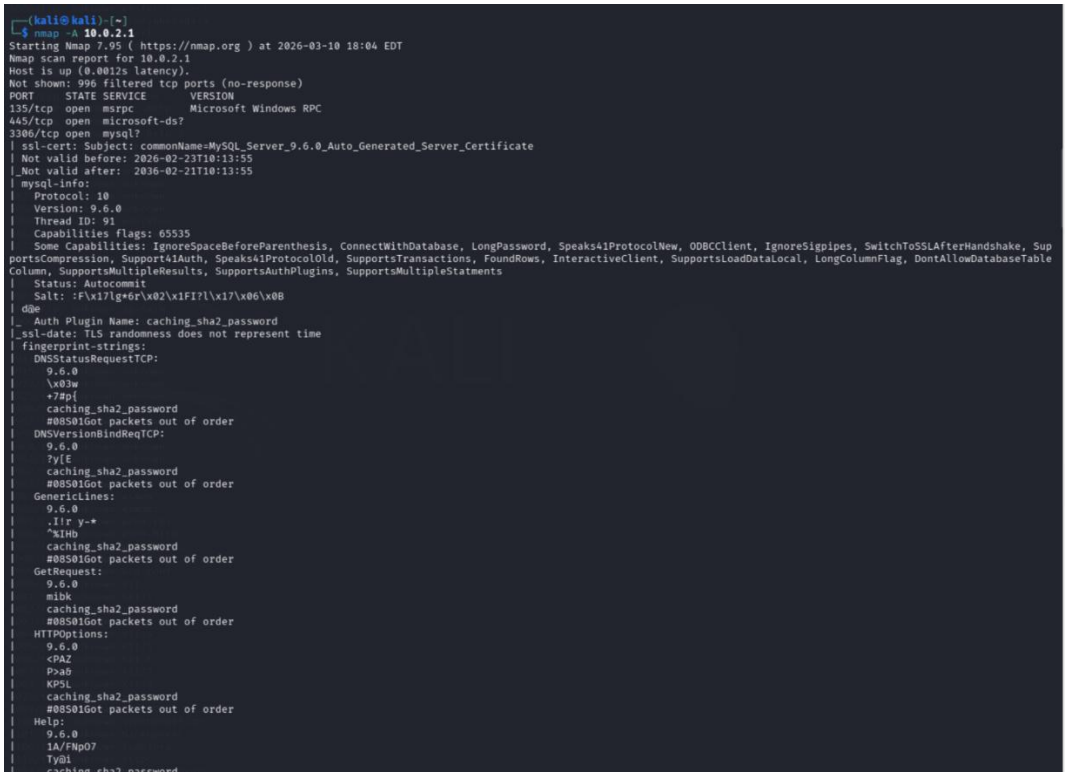
5.0 Aggressive Scanning

5.1 Objective

The lab sheet directed me to run an “Aggressive Scan”, which is a comprehensive command that combines OS detection, version detection, script scanning, and traceroute into a single process [8].

5.2 Methodology and Finding

- **Command:** nmap -A 10.0.2.1



```
┌──(kali㉿kali)-[~]
└─$ nmap -A 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:04 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0012s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
445/tcp  open  microsoft-ds?
3306/tcp open  mysql?
| ssl-cert: Subject: commonName=MySQL_Server_9.6.0_Auto_Generated_Server_Certificate
| Not valid before: 2026-02-23T10:13:55
|_Not valid after:  2036-02-21T10:13:55
| mysql-info:
|   Protocol: 10
|   Version: 9.6.0
|   Thread ID: 91
|   Capabilities flags: 65535
|   Some Capabilities: IgnoreSpaceBeforeParenthesis, ConnectWithDatabase, LongPassword, Speaks41ProtocolNew, ODBCClient, IgnoreSigpipes, SwitchToSSLAfterHandshake, Sup
portsCompression, Support41Auth, Speaks41ProtocolOld, SupportsTransactions, FoundRows, InteractiveClient, SupportsLoadDataLocal, LongColumnFlag, DontAllowDatabaseTable
Column, SupportsMultipleResults, SupportsAuthPlugins, SupportsMultipleStatments
|   Status: Autocommit
|   Salt: :F\x17lg*6r\x02\x1FI?l\x17\x06\x0B
| d@e
|_  Auth Plugin Name: caching_sha2_password
|_ssl-date: TLS randomness does not represent time
| fingerprint-strings:
|   DNSStatusRequestTCP:
|     9.6.0
|     \x03w
|     +7#p{
|     caching_sha2_password
|     #08S01Got packets out of order
|   DNSVersionBindReqTCP:
|     9.6.0
|     ?y[E
|     caching_sha2_password
|     #08S01Got packets out of order
|   GenericLines:
|     9.6.0
|     .I!r y-*
|     ^%1Hb
|     caching_sha2_password
|     #08S01Got packets out of order
|   GetRequest:
|     9.6.0
|     mibk
|     caching_sha2_password
|     #08S01Got packets out of order
|   HTTPOptions:
|     9.6.0
|     <PAZ
|     P>a6
|     KP5L
|     caching_sha2_password
|     #08S01Got packets out of order
|   Help:
|     9.6.0
|     1A/FNpO7
|     Ty@1
|     caching_sha2_password
```

Figure 11. Execution of an Aggressive Scan (-A) on 10.0.2.1

- **Finding:** This was the most data-rich activity. It not only confirmed the open port (135, 445, 3306, 5432) but also pulled security certificates. Specifically, for port 3306 (MySQL), Nmap found and automatically generated certificate valid until 2036. It also performed a traceroute, confirming the target is just one “hop” away [9].

```
MAC Address: 52:55:0A:00:02:01 (Unknown)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: VoIP adapter|bridge|general purpose
Running (JUST GUESSING): AT&T embedded (91%), Oracle Virtualbox (90%), Slirp (90%), QEMU (89%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: AT&T BGW210 voice gateway (91%), Oracle Virtualbox Slirp NAT bridge (90%), QEMU user mode network gateway (89%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|   date: 2026-03-10T22:05:27
|_  start_date: N/A
|_clock-skew: 49s
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required

TRACEROUTE
HOP RTT     ADDRESS
1   1.24 ms 10.0.2.1

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 43.45 seconds
```

Figure 12. Aggressive scan output

6.0 Nmap Scripting Engine (NSE)

6.1 Objective

In this section, the exercise practised using Nmap's built-in scripts to automate more complex tasks, such as looking for specific vulnerabilities.

6.2 Methodology and Findings

Vulnerabilities Detection: The scan ran the 'vuln' script category (nmap --script vuln 10.0.2.1). The results showed "false" of "failed to negotiate" for several common SMB vulnerabilities (like MS10-054), indicating that while the ports are open, these specific old exploits might not be applicable.

Analysis: The '--script vuln' flag directs Nmap to run its built-in category of scripts designed to identify known vulnerabilities, such as Heartbleed (SSL/TLS) and EternalBlue (SMB) [10].

In my results, the "failed to negotiate" message indicates that while the SMB ports (such as 445) are open, the specific handshake required for those older exploits failed. This suggests that the target host may have been patched against those specific vulnerabilities.

```
┌──(kali㉿kali)-[~]
└─$ nmap --script vuln 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:05 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0021s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Host script results:
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: Could not negotiate a connection:SMB: Failed to receive bytes: ERROR
|_samba-vuln-cve-2012-1182: Could not negotiate a connection:SMB: Failed to receive bytes: ERROR

Nmap done: 1 IP address (1 host up) scanned in 45.42 seconds
```

Figure 13. Running the 'vuln' script category to identify potential security weaknesses and SMB vulnerabilities

HTTP Enumeration: This exercise used nmap --script http-enum 10.0.2.1.

According to technical documentation, the http-enum script is part of a targeted web assessment suite used to enumerate directories and files specifically on active web servers [10] because my initial port scans confirmed the target host was not listening on standard web ports (80/443), the script correctly produced no output, as there was no active HTTP service to probe for web based assets.

```
┌──(kali㉿kali)-[~]
└─$ nmap --script http-enum 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:06 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0026s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.82 seconds
```

Figure 14. Attempting HTTP enumeration; results correctly return empty due to the absence of a web server on port 80

Combined Default Scripts: This scan combined version detection and OS detection with the 'default' script set (nmap -sV -O –script default 10.0.2.1). This gave a through overview of the target's security settings, such as "Message signing enabled but not required" for the SMB service.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -O --script default 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:07 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0013s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
445/tcp  open  microsoft-ds?
3306/tcp open  mysql?
| mysql-info:
|   Protocol: 10
|   Version: 9.6.0
|   Thread ID: 1667
|   Capabilities flags: 65535
|   Some Capabilities: InteractiveClient, Support41Auth, FoundRows, LongPassword, IgnoreSigpipes, Speaks41ProtocolOld, SupportsTransactions, SupportsLoadDataLocal, Ign
oreSpaceBeforeParenthesis, Speaks41ProtocolNew, SwitchToSSLAfterHandshake, ODBCClient, DontAllowDatabaseTableColumn, LongColumnFlag, SupportsCompression, ConnectWithDa
tabase, SupportsMultipleStatments, SupportsMultipleResults, SupportsAuthPlugins
|   Status: Autocommit
|   Salt: [?`-\x08J\\m]XXE\x14|N(\x0C*C
|_  Auth Plugin Name: caching_sha2_password
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=MySQL_Server_9.6.0_Auto_Generated_Server_Certificate
| Not valid before: 2026-02-23T10:13:55
|_Not valid after:  2036-02-21T10:13:55
| fingerprint-strings:
|   DNSStatusRequestTCP:
|     9.6.0
|     D?<Lq%
|     _.hP
|     caching_sha2_password
|     #08S01Got packets out of order
|   DNSVersionBindReqTCP:
|     9.6.0
|     K`m>BcfX
|     7s5j
|     jVuB1N!
|     caching_sha2_password
|     #08S01Got packets out of order
|   GenericLines, Help:
|     9.6.0
|     caching_sha2_password
|     #08S01Got packets out of order
|   GetRequest:
|     9.6.0
|     %Mty"0|iCmgB
|     caching_sha2_password
|     #08S01Got packets out of order
|   HTTPOptions:
|     9.6.0
|     j_U9
|     cs5Nw2
|     caching_sha2_password
|     #08S01Got packets out of order
|   NULL:
|     9.6.0
|     caching_sha2_password
|   RPCCheck:
|     9.6.0
|     DX<E
```

Figure 15. Combining version and OS detection with default scripts to audit SMB message signing and security settings

7.0 Output Options

7.1 Objective

The task was to practice saving these results in different formats, which is essential for professional reporting and record-keeping.

7.2 Methodology and Findings

Text File (-oN): Saved the standard terminal output to output.txt for easy reading [1].

```
┌──(kali㉿kali)-[~]
└─$ nmap -oN output.txt 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:08 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0032s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.51 seconds
```

Figure 16. Saving scan results to a standard text file (-oN) for human-readable documentation

XML Format (-oX): Created output.xml, which is useful if I wanted to import the data into another security tool or a database [1].

```
┌──(kali㉿kali)-[~]
└─$ nmap -oG output.gerp 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:10 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0020s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 6.74 seconds
```

Figure 17. Generating an XML output file (-oX) for structured data integration with other security tools

Greppable Output (-oG): Generated output.grep, which allows a user to quickly search through large scan results using command-line tools like 'grep'

```
┌──(kali㉿kali)-[~]
└─$ nmap -oG output.gerp 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:10 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0020s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 6.74 seconds
```

Figure 18. Creating a greppable output file (-oG) to facilitate rapid searching of scan data via command-line utilities

8.0 Advanced Scanning Techniques

8.1 Objective

This section of the lab sheet focused on advances network auditing techniques. These tasks were designed to demonstrate how a security professional can adapt Nmap scans to bypass firewalls, mask their identity, or optimise scan performance on a complex network.

8.2 Methodology and Findings

Firewall Evasion (Decoys and Fragmentation):

```
┌──(kali㉿kali)-[~]
└─$ nmap -D RND:10 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 18:10 EDT
Warning: 10.0.2.1 giving up on port because retransmission cap hit (10).

┌──(kali㉿kali)-[~]
└─$ nmap -Pn 10 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:16 EDT
Nmap scan report for 10 (0.0.0.10)
Host is up (0.0100s latency).
All 1000 scanned ports on 10 (0.0.0.10) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap scan report for 10.0.2.1
Host is up (0.0024s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 2 IP addresses (2 hosts up) scanned in 6.06 seconds
```

Figure 19. Implementing a Decoy scan (-D) combined with the -Pn flag; to bypass host discovery (ping) to treat the target as online, while the retransmission cap indicates the target is dropping the subset spoofed traffic

I experimented with two primary methods for evading detection:

Decoys (-D RND:10): This exercise attempted to hide my IP address among ten random spoofed addresses [1].

No Ping (-Pn): To ensure the scan proceeded even if the target was configured to ignore ICMP echo requests (ping), the exercise used the -Pn flag to skip the initial host discovery phase and treat the target as "online" [1].

- **Result:** Nmap hit a "retransmission cap". This was a significant finding as it suggested that the target's security systems detected the unusual traffic and began dropping packets, demonstrating an active defence mechanism.

Fragmentation (-f): This activity involved splitting the IP packets into smaller pieces to confuse simple packet filters.

```
┌──(kali㉿kali)-[~]
└─$ nmap -f 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:40 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0035s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 6.03 seconds
```

Figure 20. Utilizing packet fragmentation (-f) to successfully bypass simple filters and verify open ports on host 10.0.2.1

- **Result:** Unlike the decoy scan, which focuses on obfuscating the source, fragmentation successfully bypassed the basic filtering layer [1]. This scan was completed without being dropped or blocked, confirming the standard open ports (135, 445, 3306, 5432).

Identify Masking (MAC Spoofing):

The lab required me to spoof my hardware address to appear as a different device.

- **Command:** nmap --spoof-mac 00:11:22:33:44:55 10.0.2.1

```
┌──(kali㉿kali)-[~]
└─$ nmap --spoof-mac 00:11:22:33:44:55 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:41 EDT
Spoofing MAC address 00:11:22:33:44:55 (Cimsys)
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.57 seconds
```

Figure 21. Attempting MAC Address spoofing; the "Host seems down" result suggests the presence of MAC-based filtering or port security

- **Result:** The host appeared as "down". This indicates that the target network likely has port security or MAC filtering that blocks unrecognised hardware addresses from communicating.

Scanning Efficiency (Timing and Randomisation):

Randomisation (-r): By scanning hosts in a random order, the scan confirmed three active hosts (10.0.2.1, 10.0.2.2, and 10.0.2.15).

```
┌──(kali㉿kali)-[~]
└─$ nmap -r 10.0.2.1-255
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:42 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0024s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap scan report for 10.0.2.2
Host is up (0.00068s latency).
All 1000 scanned ports on 10.0.2.2 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:D5:1D:7D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 10.0.2.15
Host is up (0.0000020s latency).
All 1000 scanned ports on 10.0.2.15 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 255 IP addresses (3 hosts up) scanned in 7.97 seconds
```

Figure 22. Executing a randomised host scan (-r) across the 10.0.2.1-225 range to identify active nodes

Idle Scan (-sI): The command nmap -sI zombie_host 10.0.2.1 was executed. The "zombie" host makes the scan to appears to originate from a less suspicious source [11].

The scan resulted in an error because "zombie_host" is a placeholder name.

For this scan to function, a reachable IP address of an idle host with a predictable IP ID sequence is required [12]. This activity demonstrated that advanced stealth scans require a pre-configured infrastructure (a functional zombie) to successfully mask the attacker's identity.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sI zombie_host 10.0.2.1
WARNING: Many people use -Pn w/Idlescan to prevent pings from their true IP.  On the other hand, timing info Nmap gains from pings can allow for faster, more reliable
scans.
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:43 EDT
Could not resolve idle scan zombie host "zombie_host": No address associated with hostname
QUITTING!
```

Figure 23. Executing an Idle Scan (-sI) using a placeholder

Timing (-T4): The exercise used the aggressive timing template to speed up the process. According to official Nmap documentation, this template is recommended for use on reasonably modern and reliable networks, as it makes several internal adjustments to speed up the process, such as capping the dynamic scan delay at 10 ms for TCP ports, without significantly sacrificing accuracy [13]

- **Finding:** The scan was highly efficient, completing in approximately 5 seconds.

```
┌──(kali㉿kali)-[~]
└─$ nmap -T4 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:45 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0031s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 5.09 seconds
```

Figure 24. Using the aggressive timing template (-T4) to optimise scan speed while maintaining data accuracy

Analysis: While higher templates like -T5 (Insane) exist, they can be too aggressive and may lead to packet loss or missing open ports. Using -T4 provided a "sweet spot" for this virtual environment, allowing for rapid results while maintaining the integrity of the port discovery [13]

9.0 Other Useful Commands

9.1 Objective

This final set of activities covered specialised Nmap features used for network path analysis and protocol-specific auditing.

9.2 Methodology and Findings

Traceroute

- **Command:** nmap --traceroute 10.0.2.1

```
┌──(kali㉿kali)-[~]
└─$ nmap --traceroute 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:45 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0025s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

TRACEROUTE
HOP RTT     ADDRESS
1   2.49 ms 10.0.2.1

Nmap done: 1 IP address (1 host up) scanned in 5.87 seconds
```

Figure 25. Performing a network traceroute to confirm the target's physical proximity (one hop) within the virtual environment

- **Result:** The output showed a single hop with a round-trip time (RTT) of 2.49 ms, confirming the target is on the immediate local virtual network.
- **Analysis:** This result confirms that the target is on the immediate local virtual network, with no intermediate routers or gateways between the two hosts. By using this feature, I was able to verify the physical proximity of the target within the virtual environment, ensuring that the low latency and high reliability of the subsequent scans were due to this direct "one-hop" connection [14].

IPv6 Scanning

- **Command:** nmap -6 [2001:0db8::1]

```
┌──(kali㉿kali)-[~]
└─$ nmap -6 [2001:0db8::1]
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:46 EDT
Failed to resolve "[2001:0db8::1]".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.04 seconds
```

Figure 26. Attempting an IPv6 scan; the failure highlights a lack of global IPv6 connectivity on the local link Link-Local interface

```
┌──(kali㉿kali)-[~]
└─$ ip -6 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN qlen 1000
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 fe80::e63c:89b6:7983:517b/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure 27. Verifying local IPv6 configuration using the 'ip -6 addr' command

- **Result:** This command failed to resolve. By running 'ip -6 addr', I discovered that my virtual machine has a "Link-Local" address (fe80::...) and no global IPv6 connectivity, which is why it could not reach the external documentation address

TCP ACK Scan

- **Command:** nmap -sA 1.0.2.1

```
┌──(kali㉿kali)-[~]
└─$ nmap -sA 10.0.2.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:52 EDT
Nmap scan report for 10.0.2.1
Host is up (0.00029s latency).
All 1000 scanned ports on 10.0.2.1 are in ignored states.
Not shown: 1000 unfiltered tcp ports (reset)
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.55 seconds
```

Figure 28. Executing a TCP ACK scan (-sA) to audit firewall statefulness

- **Result:** All 1000 ports were reported as "unfiltered". This confirms the absence of a stateful firewall; the target is responding with 'reset' (RST) packets, including that the ports are reachable even if the services are protected [15].

Filtering for Open Ports

- **Command:** nmap --open 10.0.2.10-225

```
┌──(kali㉿kali)-[~]
└─$ nmap --open 10.0.2.1-225
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 19:53 EDT
Nmap scan report for 10.0.2.1
Host is up (0.0017s latency).
Not shown: 996 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT     STATE SERVICE
135/tcp  open  msrpc
445/tcp  open  microsoft-ds
3306/tcp open  mysql
5432/tcp open  postgresql
MAC Address: 52:55:0A:00:02:01 (Unknown)

Nmap done: 225 IP addresses (3 hosts up) scanned in 8.07 seconds
```

Figure 29. Applying the --open filter to a range scan (10.0.2.10-225) to generate a concise report of active services only

- **Result:** This was an effective way to clean up the report. It ignored all closed and filtered ports, providing a concise list of the four active services on 10.0.2.1 [15].

Final Reflection

Completing this comprehensive lab exercise has provided a clear picture of the target environment's security posture. While the target machine (10.0.2.1) has several open and potentially vulnerable database services, it also displays active defensive traits-specifically observed when it blocked my decoy scans and MAC spoofing attempts.

The most valuable takeaway was the troubleshooting process. Identifying why the IPv6 scan failed (lack of global addressing) and why the decoy scan timed out (security suppression) is just as important as the scan results themselves. This lab has demonstrated that Nmap is a dual-purpose tool: it is as much about understanding the network's defences as it is about identifying open ports.

Secure Shell (SSH)

SSH Connection Basics

The focus of this section is on Secure Shell (SSH).

Objective

To establish a secure remote session between the 'WordPress' server and a target host, exploring both successful connections and common network errors.

Methodology and Findings

Establishing a Standard Connection: The process initiated a connection to the remote host at 10.0.2.4 using the standard SSH command

- **Command:** ssh student@10.0.2.4

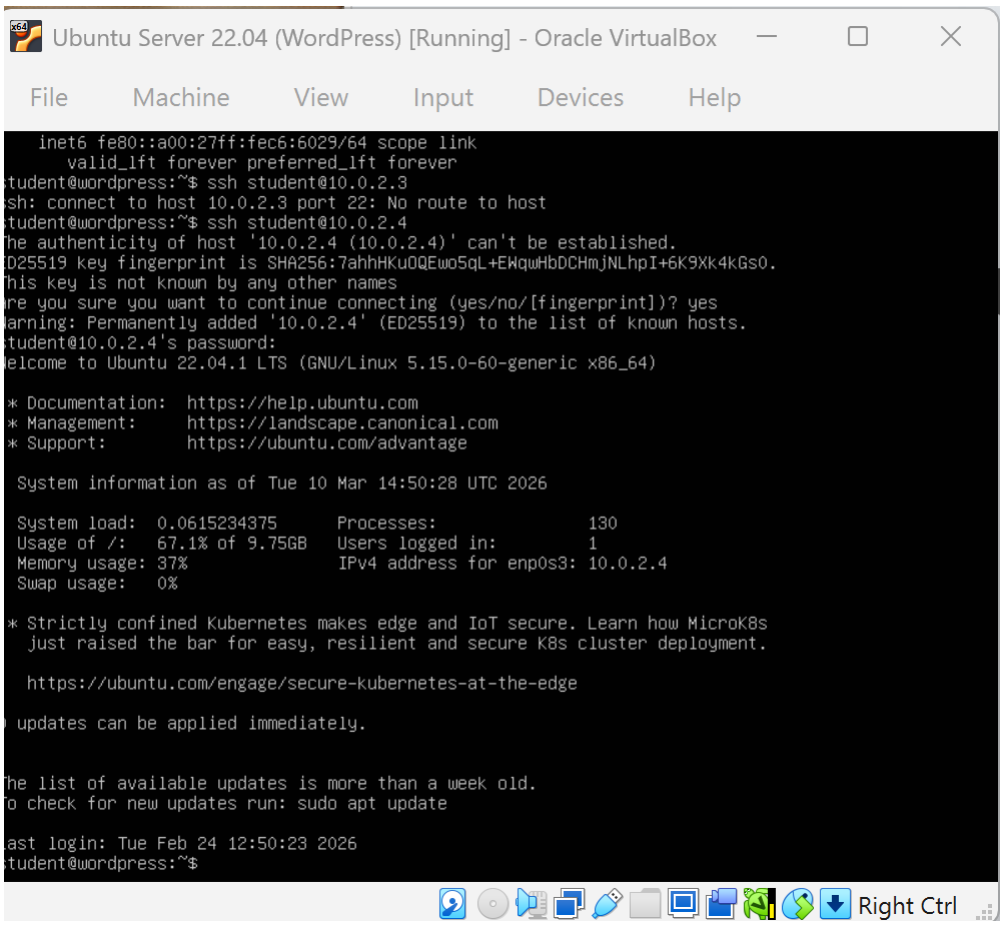


Figure 30. Initial SSH connection to 10.0.2.4

- **Observation:** Upon the first connection, the system presented an ED25519 key fingerprint. This is a security feature used to verify the authenticity of the host and prevent Man-in-the-Middle (MitM) attacks. After confirming 'yes' and entering the password, the exercise successfully accessed the remote Ubuntu 22.04 environment [16].

Troubleshooting Port Connectivity: The lab activity required testing connections on non-standard ports to observe system responses.

Non-existent Port: I attempted to connect via ports 2222 and 9876.

```
Last login: Tue Feb 24 12:50:23 2026
student@wordpress:~$ ssh -p 2222 student@10.0.2.4
ssh: connect to host 10.0.2.4 port 2222: Connection refused
student@wordpress:~$ ssh student@10.0.2.4 -p 2222
ssh: connect to host 10.0.2.4 port 2222: Connection refused
student@wordpress:~$ ssh student@10.0.2.4 -p 9876
ssh: connect to host 10.0.2.4 port 9876: Connection refused
student@wordpress:~$ sudo ss -ltn
[sudo] password for student:
State      Recv-Q     Send-Q         Local Address:Port           Peer Address:Port     Process
LISTEN     0          128                  0.0.0.0:22                0.0.0.0:*
LISTEN     0          70                 127.0.0.1:33060             0.0.0.0:*
LISTEN     0          151                  0.0.0.0:3306              0.0.0.0:*
LISTEN     0          4096           127.0.0.53%lo:53                0.0.0.0:*
LISTEN     0          128                     [::]:22                   [::]:*
LISTEN     0          511                        *:80                      *:*
student@wordpress:~$ _
```

Figure 31. Troubleshooting port connectivity; attempts to connect via non-standard ports (2222 and 9876) resulted in "Connection refused" errors

- **Finding:** Both attempts returned a “Connection refused” error. Using the command sudo ss -ltn (Socket Statistics), I verified that the server was only listening for SSH traffic on the default Port 22.

Protocol Mismatch: I attempted to connect to port 3306 (the MySQL port)

```
ssh student@10.0.2.4 -p 3306
kex_exchange_identification: banner line contains invalid characters
banner exchange: Connection to 10.0.2.4 port 3306: invalid format
student@wordpress:~$ ssh -p 3306 student@10.0.2.4
kex_exchange_identification: banner line contains invalid characters
banner exchange: Connection to 10.0.2.4 port 3306: invalid format
student@wordpress:~$ ssh -p 22 student@10.0.2.4
student@10.0.2.4's password:
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 16:49:19 UTC 2026

  System load:  0.0458984375      Processes:               132
  Usage of /:   67.3% of 9.75GB   Users logged in:         1
  Memory usage: 36%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 14:50:29 2026 from 10.0.2.4
student@wordpress:~$
```

Figure 32. Attempting an SSH connection

- **Finding:** This resulted in a “banner exchange” error. This was a valuable finding, as it demonstrated that while the port was open, the SSH client could not communicate with a database service, highlighting the protocol-specific nature of SSH.

2.0 Authentication and Key Management

2.1 Objective

To move beyond basic password authentication by generating and installing cryptographic key pairs, thereby enabling more secure “passwordless” logins.

2.2 Methodology and Findings

Generating the RSA Key Pair: Following the lab instructions, the scan generated a high-security 4096-bit RSA key pair.

- **Command:** ssh-keygen -t rsa -b 4096 -C

```
student@wordpress:~$ ssh-keygen -t rsa -b 4096 -C "student@example.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/student/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/student/.ssh/id_rsa
Your public key has been saved in /home/student/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:X9kHqMXetvGiUtFEVLCStv5qrVBAidFMh8+BnPdQMSc student@example.com
The key's randomart image is:
+---[RSA 4096]----+
|         .Bo=.+E+o|
|         ..Oo+o.= |
|           .+B*o  |
|            *+*o. |
|         S . *.= .|
|          . +.. = |
|           o...o .|
|           ...o.. |
|            o+o.  |
+----[SHA256]-----+
student@wordpress:~$ _
```

Figure 33. Generating a 4096-bit RSA key pair using 'ssh-keygen', including the creation of a unique "randomart" fingerprint for visual verification

Objective: This created two files: a private key (id_rsa) and a public key (id_rsa.pub). The process produced a “randomart” image, which is visual representation of the key’s fingerprint [16].

Deploying the Public Key: To enable the remote server to recognise the local machine without a password, I transferred the public key to the target host.

- **Command:** ssh-copy-id student@10.0.2.3

```
student@wordpress:~$ ssh-copy-id student@10.0.2.4
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/student/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are alr
eady installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to inst
all the new keys
student@10.0.2.4's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'student@10.0.2.4'"
and check to make sure that only the key(s) you wanted were added.

student@wordpress:~$ _
```

Figure 34. Using ssh-copy-id to securely deploy the public key (id_rsa.pub) to the remote host at 10.0.2.3

- **Finding:** The utility successfully installed the local key into the remote server's authorized_keys file. A final password prompt was required to authorise this installation, after which the system confirmed: "Number of Key(s) added: 1".

3.0 Managing SSH Sessions

3.1 Objective

To practice executing remote commands without establishing an interactive shell and learning how to properly terminate secure sessions

3.2 Methodology and Findings

Remote Command Execution: I executed a directory listing command on the remote host (10.0.2.4) directly from my local terminal.

- **Command:** ssh student@10.0.2.4 ls -la

```
student@wordpress:~$ ssh student@10.0.2.4 ls -la
total 88
drwxr-x--- 6 student student 4096 Feb 10 11:08 .
drwxr-xr-x 3 root    root    4096 Oct 11  2022 ..
-rw------- 1 student student 3459 Feb 14  2023 .bash_history
-rw-r--r-- 1 student student  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 student student 3771 Jan  6  2022 .bashrc
drwx------ 2 student student 4096 Oct 11  2022 .cache
-rw------- 1 root    root    5208 Feb 10 00:26 file-enc-privkey..asc
-rw-r--r-- 1 root    root    2452 Feb 10 00:23 file-enc-pubkey.txt
-rw-r--r-- 1 root    root      45 Feb  9 23:59 filess.tar.gz
-rw-r--r-- 1 root    root     438 Feb 10 00:10 files.tar.gz
-rw-r--r-- 1 root    root     902 Feb 10 00:31 files.tar.gz.gpg
drwx------ 2 student student 4096 Feb 10 00:21 .gnupg
drwxrwxr-x 3 student student 4096 Feb  9 23:49 .local
-rw------- 1 student student  962 Feb 14  2023 .mysql_history
-rw-r--r-- 1 root    root      13 Feb  9 23:53 output.txt
-rw-r--r-- 1 student student  807 Jan  6  2022 .profile
drwx------ 2 student student 4096 Mar 10 16:56 .ssh
-rw-r--r-- 1 student student    0 Oct 11  2022 .sudo_as_admin_successful
-rw-rw-r-- 1 student student   75 Feb 10 11:08 test2.txt
-rw-r--r-- 1 root    root       4 Feb 10 10:56 test.txt
-rw-r--r-- 1 root    root      13 Feb  9 23:49 text.txt
-rw-r--r-- 1 root    root      91 Feb  9 23:51 text.txt.gpg
student@wordpress:~$
```

Figure 35. Detailed list of files on the remote server

- **Finding:** The command successfully returned a detailed list of files on the remote server, including hidden files such as .bash_history and .ssh. This confirmed that the SSH key authentication configured in the previous part was working correctly, as the command executed and returned data seamlessly.

Terminating Sessions

- **Command:** exit

```
student@wordpress:~$ ssh student@10.0.2.4 ls -la
total 88
drwxr-x--- 6 student student 4096 Feb 10 11:08 .
drwxr-xr-x 3 root    root    4096 Oct 11  2022 ..
-rw------- 1 student student 3459 Feb 14  2023 .bash_history
-rw-r--r-- 1 student student  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 student student 3771 Jan  6  2022 .bashrc
drwx------ 2 student student 4096 Oct 11  2022 .cache
-rw------- 1 root    root    5208 Feb 10 00:26 file-enc-privkey..asc
-rw-r--r-- 1 root    root    2452 Feb 10 00:23 file-enc-pubkey.txt
-rw-r--r-- 1 root    root      45 Feb  9 23:59 filess.tar.gz
-rw-r--r-- 1 root    root     438 Feb 10 00:10 files.tar.gz
-rw-r--r-- 1 root    root     902 Feb 10 00:31 files.tar.gz.gpg
drwx------ 2 student student 4096 Feb 10 00:21 .gnupg
drwxrwxr-x 3 student student 4096 Feb  9 23:49 .local
-rw------- 1 student student  962 Feb 14  2023 .mysql_history
-rw-r--r-- 1 root    root      13 Feb  9 23:53 output.txt
-rw-r--r-- 1 student student  807 Jan  6  2022 .profile
drwx------ 2 student student 4096 Mar 10 16:56 .ssh
-rw-r--r-- 1 student student    0 Oct 11  2022 .sudo_as_admin_successful
-rw-rw-r-- 1 student student   75 Feb 10 11:08 test2.txt
-rw-r--r-- 1 root    root       4 Feb 10 10:56 test.txt
-rw-r--r-- 1 root    root      13 Feb  9 23:49 text.txt
-rw-r--r-- 1 root    root      91 Feb  9 23:51 text.txt.gpg
student@wordpress:~$ exit
logout
Connection to 10.0.2.4 closed.
student@wordpress:~$
```

Figure 36. Typing 'exit' to close the connection of 10.0.2.4

- **Observation:** Upon typing exit, the terminal confirmed “Connection to 10.0.2.4 closed,” demonstrating the proper way to end a cryptographic session and free up system resources.

4.0 File Transfer with SCP (Secure Copy)

4.1 Objective

To use the Secure Copy Protocol to move files between the local machine and the remote server.

4.2 Methodology and Findings

Uploading Files: I transferred a local file named test2.txt to the remote student home directory.

- **Command:** scp test2.txt student@10.0.2.4:/home/student/

```
student@wordpress:~$ scp test2.txt student@10.0.2.4:/home/student/
test2.txt                                                     100%   75    39.2KB/s   00:00
student@wordpress:~$
```

Figure 37. Using SCP to upload 'test2.txt' to the remote host

- **Finding:** The terminal provided a progress bar showing 100% completion at a rate of 39.2KB/s. I verified the transfer by running the remote ls -la command again, which showed test2.txt present on the remote host with a timestamp of 10 February.

```
student@wordpress:~$ scp student@10.0.2.4:/home/student/test2.txt
usage: scp [-346ABCOpqRrsTv] [-c cipher] [-D sftp_server_path] [-F ssh_config]
           [-i identity_file] [-J destination] [-l limit]
           [-o ssh_option] [-P port] [-S program] source ... target
student@wordpress:~$
```

Figure 38. Encountering an SCP usage error

Command Syntax Awareness: I attempted to download a file but initially encountered a usage error by not providing a local destination path. This served as a practical reminder that SCP requires both a source and a destination to function.

5.0 Secure File Transfer with SFTP

5.1 Objective

To engage in an interactive, FTP-like secure session for more complex file management tasks.

5.2 Methodology and Findings

Interactive Connections: I established a connection using the Secure File Transfer Protocol.

- **Command:** sftp student@10.0.2.4

- **Finding:** The prompt changed to sftp>, indicating an active interactive session Remote Navigation and Retrieval:

  Inside the session, I used the ls command to view files such as output.txt, files.tar.gz, and text.txt.gpg.

```
student@wordpress:~$ sftp student@10.0.2.4
Connected to 10.0.2.4.
sftp> ls
file-enc-privkey..asc    file-enc-pubkey.txt      files.tar.gz             files.tar.gz.gpg
filess.tar.gz            output.txt               test.txt                 test2.txt
text.txt                 text.txt.gpg
sftp> _
```

Figure 39. Establishing an interactive SFTP session with 10.0.2.4 and using the 'ls' command to enumerate remote files for management

Downloading Files: I used the get command (get test2.txt) to pull the file from the remote server back to my local directory.

```
sftp> get test2.txt
Fetching /home/student/test2.txt to test2.txt
test2.txt                                                    0%    0     0.0KB/s   --:-- ETA
sftp>
```

Figure 40. Executing the 'get' command within the SFTP interface to securely download 'test2.txt' from the remote server to the local machine

- **Observation:** SFTP provided a clear interface for managing multiple files without having to re-authenticate for every individual transfer.

6.0 Tunneling with SSH

6.1 Objective

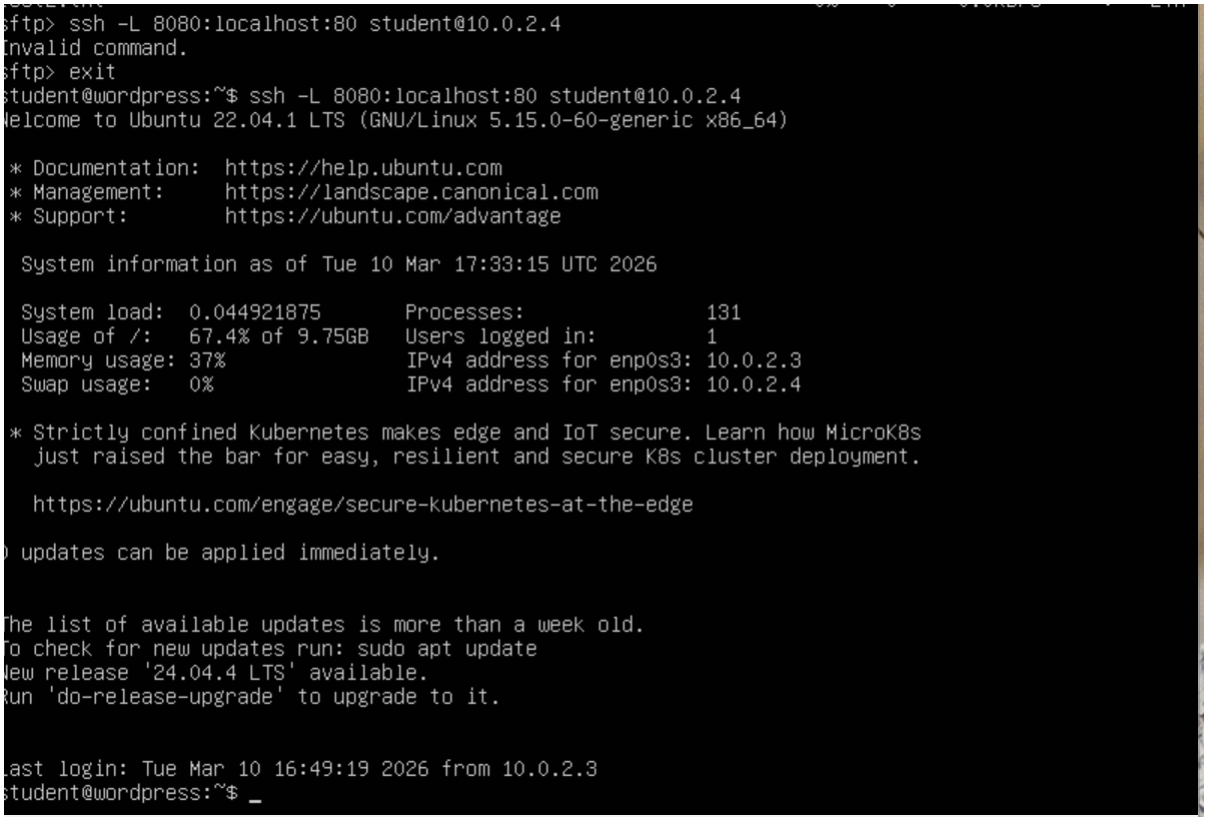
The primary objective was to configure encrypted 'tunnels' to relay service ports securely between a local client and a remote server.

This technique is essential for transporting unencrypted data such as FTP or VNC through a secure channel or bypassing intermediate firewalls [17].

6.2 Methodology and Findings

Local Port Forwarding: I configured the SSH client to listen on a local port and tunnel any connection to a specified port on the remote SSH server [17].

- **Command:** ssh -L 8080:localhost:80 student@10.0.2.4



```
sftp> ssh -L 8080:localhost:80 student@10.0.2.4
Invalid command.
sftp> exit
student@wordpress:~$ ssh -L 8080:localhost:80 student@10.0.2.4
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 17:33:15 UTC 2026

  System load:  0.044921875       Processes:               131
  Usage of /:   67.4% of 9.75GB   Users logged in:         1
  Memory usage: 37%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 16:49:19 2026 from 10.0.2.3
student@wordpress:~$ _
```

Figure 41. Setting up Local Port Forwarding (-L); the initial "Invalid command" error highlights the necessity of executing tunneling commands from the local shell rather than an active SFTP session

- **Finding:** I attempted to forward traffic from the local machine's port 8080 to the remote server's web service on port 80.
- **Observation:** As seen in Figure 41, I initially encountered an "Invalid command" error. This occurred because I attempted to execute the forwarding command within an active SFTP session. Technical documentation clarifies that tunnelling must be initiated from the local shell to establish the bridge before a session begins. Once corrected, the tunnel was established successfully.

Remote Port Forwarding: I practiced the reverse process, forwarding traffic from port 9090 on the remote machine back to my local machine [17].

- **Command:** ssh -R 9090:localhost:80 student@10.0.2.4

```
student@wordpress:~$ ssh -R 9090:localhost:80 student@10.0.2.4
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 17:35:35 UTC 2026

  System load:  0.09423828125     Processes:               135
  Usage of /:   67.4% of 9.75GB   Users logged in:         1
  Memory usage: 37%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 17:33:16 2026 from 10.0.2.3
student@wordpress:~$
```

Figure 42. Establishing a Remote Port Forward (ssh -R) to bridge traffic from the remote server back to a local service on port 80

- **Finding:** The connection was established without error, theoretically allowing any user on the remote server to access my local web service through that encrypted bridge.
- **Analysis:** This setup effectively bridges traffic from the remote environment back to a local service. In a professional context, this is a powerful tool for exposing a local development server to a remote machine through an encrypted, secure bridge, even if the local machine is behind a NAT or firewall.

7.0 Multiplexing and Persistent Connections

7.1 Objective

The objective of this activity was to implement SSH Multiplexing using the ControlMaster directive. This allows multiple SSH sessions, as well as scp or sftp requests, to reuse a single established TCP connection, significantly reducing the overhead required to negotiate new secure connections [18].

7.2 Methodology and Findings

- **Command:** ssh -o ControlMaster=auto -o ControlPath=~/.ssh/ssh_mux_%h_%p_%r -o ControlPersist=60s student@10.0.2.4

```
student@wordpress:~$ ssh -o ControlMaster=auto -o ControlPath=~/.ssh/ssh_mux_%h_%p_%r -o ControlPers
ist=60s student@10.0.2.4
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 17:40:31 UTC 2026

  System load:  0.001953125       Processes:               139
  Usage of /:   67.4% of 9.75GB   Users logged in:         1
  Memory usage: 37%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 17:35:35 2026 from 10.0.2.3
student@wordpress:~$
```

Figure 43. Implementing SSH Multiplexing with ControlMaster; the configuration allows multiple sessions to share a single TCP connection, reducing handshake overhead

- **Finding:** By setting ControlPersist=60s, the master connection remains open in the background even after I log out. This significantly reduces the overhead and "handshake" time for subsequent logins within that minute, which is vital for automated scripts and fast-paced administration [18].

8.0 Advanced SSH Options and Debugging

8.1 Objective

To use diagnostic tools and performance limiters to manage secure connections more effectively.

8.2 Methodology and Findings

Custom Key Usage: I attempted to use a specific private key with ssh -i ~/.ssh/id_rsa_custom.

```
student@wordpress:~$ ssh -i ~/.ssh/id_rsa_custom student@10.0.2.4
Warning: Identity file /home/student/.ssh/id_rsa_custom not accessible: No such file or directory.
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 17:42:00 UTC 2026

  System load:  0.080078125       Processes:               143
  Usage of /:   67.4% of 9.75GB   Users logged in:         1
  Memory usage: 37%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 17:40:31 2026 from 10.0.2.3
student@wordpress:~$ _
```

Figure 44. Troubleshooting custom identity files (-i); the error message confirms the SSH client's behaviour when a specified private key is inaccessible or missing

- **Finding:** The system returned a warning: "Identity file... not accessible". This was a useful finding as it showed that the SSH client defaults back to standard authentication methods if the specified key file is missing.

Verbose Debugging (-v): I enabled verbose mode to see the internal protocol handshake.

```
debug1: client_input_hostkeys: hostkeys file /home/student/.ssh/known_hosts2 does not exist
debug1: client_input_hostkeys: no new or deprecated keys from server
debug1: Remote: /home/student/.ssh/authorized_keys:1: key options: agent-forwarding port-forwarding
pty user-rc x11-forwarding
debug1: Remote: /home/student/.ssh/authorized_keys:1: key options: agent-forwarding port-forwarding
pty user-rc x11-forwarding
debug1: Sending environment.
debug1: channel 0: setting env LANG = "en_GB.UTF-8"
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-60-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Tue 10 Mar 17:42:00 UTC 2026

  System load:  0.080078125       Processes:               143
  Usage of /:   67.4% of 9.75GB   Users logged in:         1
  Memory usage: 37%               IPv4 address for enp0s3: 10.0.2.3
  Swap usage:   0%                IPv4 address for enp0s3: 10.0.2.4

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Tue Mar 10 17:42:01 2026 from 10.0.2.3
student@wordpress:~$ _
```

Figure 45. Enabling Verbose Mode (-v) to audit the protocol handshake, showing local environment variables and host key verification steps

- **Observation:** The debug output provided deep visibility, showing that the system was checking known_hosts2 and verifying environment variables like LANG="en_GB.UTF-8". This is the primary method used to troubleshoot why a connection might be failing at the encryption level.

Bandwidth Limiting with SCP: I practiced limiting the transfer speed of a file to prevent network congestion.

- **Command:** scp -l 1000 test2.txt student@10.0.2.4:/home/student/

```
student@wordpress:~$ scp -l 1000 test2.txt student@10.0.2.4:/remote/path
scp: /remote/path: No such file or directory
student@wordpress:~$ scp -l 1000 test2.txt student@10.0.2.4:/home/student/
test2.txt                                                     100%    0     0.0KB/s   00:00
student@wordpress:~$
```

Figure 46. Implementing bandwidth limiting (-l) during an SCP transfer and verifying system behaviour when attempting to write to non-existent remote directories

- **Finding:** While the file was very small (resulting in 0.0KB/s display), the syntax was confirmed. I also observed that trying to send a file to a non-existent directory (/remote/path) resulted in a correct "No such file or directory".

Final Reflection

Through this entire series of lab activities, I have transitioned from an external auditor (using Nmap to find the target and its open ports) to a trusted administrator (using SSH to manage the target securely).

I have demonstrated the ability to:

- **Map the attack surface:** Using Nmap to identify services and evade basic detection.
- **Secure the entryway:** Replacing weak passwords with a 4096-bit RSA key
- **Manage data and traffic:** Using SCP/SFTP for data movement and SSH Tunneling to manipulate network traffic flow safely.

The errors encountered during the lab, such as the "invalid command" in SFTP or the "missing identity file error", were critical learning points that underscored the importance of environment awareness and proper syntax in a professional cybersecurity context.

References

[1] M. Kumar, "Nmap: The Complete Guide From Beginner to Advanced," *Geek Institute Blog*, Nov. 2024. [Online]. Available: https://blog.geekinstitute.org/2024/11/nmap-network-mapping-and-security-scanning.html. [Accessed: Mar. 11, 2026].

[2] GeeksforGeeks, "What is Transmission Control Protocol (TCP)?," Feb. 11, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/what-is-transmission-control-protocol-tcp/. [Accessed: Mar. 11, 2026].

[3] GeeksforGeeks, "Port Scanning Techniques by Using Nmap," Feb. 11, 2026. [Online]. Available: https://www.geeksforgeeks.org/ethical-hacking/port-scanning-techniques-by-using-nmap/. [Accessed: Mar. 11, 2026].

[4] GeeksforGeeks, "User Datagram Protocol (UDP)," Feb. 11, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/user-datagram-protocol-udp/. [Accessed: Mar. 11, 2026].

[5] G. Lyon, "UDP Scan (-sU)," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/scan-methods-udp-scan.html. [Accessed: Mar. 11, 2026].

[6] Siberoloji, "Detecting Open, Closed, Filtered, and Unfiltered Ports with Nmap," *Siberoloji*, 2024. [Online]. Available: https://www.siberoloji.com/detecting-open-closed-filtered-and-unfiltered-ports-with-nmap/. [Accessed: Mar. 11, 2026].

[7] G. Lyon, "Remote OS Detection," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/man-os-detection.html. [Accessed: Mar. 11, 2026].

[8] Codecademy, "Nmap Aggressive Scan," *Codecademy Docs*, 2025. [Online]. Available: https://www.codecademy.com/resources/docs/cybersecurity/nmap/aggressive-scan. [Accessed: Mar. 11, 2026].

[9] M. Shivanandhan, "What is Nmap and How to Use it – A Tutorial for the Greatest Scanning Tool of All Time," *freeCodeCamp*, Oct. 2020. [Online]. Available: https://www.freecodecamp.org/news/what-is-nmap-and-how-to-use-it-a-tutorial-for-the-greatest-scanning-tool-of-all-time/. [Accessed: Mar. 11, 2026].

[10] CyberDesserts, "Nmap NSE – Nmap Scripting Engine," *CyberDesserts Blog*, 2025. [Online]. Available: https://blog.cyberdesserts.com/nmap-nse-scripting-engine/. [Accessed: Mar. 11, 2026].

[11] LabEx, "How to Perform Stealthy Nmap Scans to Avoid Detection," *LabEx Tutorials*, 2025. [Online]. Available: https://labex.io/tutorials/nmap-how-to-perform-stealthy-nmap-scans-to-avoid-detection-in-cybersecurity-415611. [Accessed: Mar. 11, 2026].

[12] G. Lyon, "TCP Idle Scan (-sI)," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/idlescan.html. [Accessed: Mar. 11, 2026].

[13] G. Lyon, "Timing Templates (-T)," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/performance-timing-templates.html. [Accessed: Mar. 11, 2026].

[14] G. Lyon, "Nmap Scan Phases," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/nmap-phases.html. [Accessed: Mar. 11, 2026].

[15] G. Lyon, "Port Scanning Options," *Nmap Network Scanning (Official Guide)*, 2026. [Online]. Available: https://nmap.org/book/port-scanning-options.html. [Accessed: Mar. 11, 2026].

[16] M. Boelen, "Using Ed25519 OpenSSH keys instead of DSA, RSA, or ECDSA," *Linux Audit*, 2024. [Online]. Available: https://linux-audit.com/ssh/using-ed25519-openssh-keys-instead-of-dsa-rsa-ecdsa/. [Accessed: Mar. 11, 2026].

[17] D. Panovski, "How to Set up SSH Tunneling (Port Forwarding)," *Linuxize*, Jan. 29, 2026. [Online]. Available: https://linuxize.com/post/how-to-setup-ssh-tunneling/. [Accessed: Mar. 11, 2026].

[18] V. Gite, "How To Reuse SSH Connection to Speed Up Remote Login Process Using Multiplexing," *nixCraft*, Jan. 16, 2025. [Online]. Available: https://www.cyberciti.biz/faq/linux-unix-reuse-openssh-connection/. [Accessed: Mar. 11, 2026].